

ML2 - HW3

Karin 207413360

Omer 322836180

Theory - Neural Network Expressivity

1. Failure of L^∞ Approximation

Let $f : [0, 1] \rightarrow \mathbb{R}$ be defined as:

$$f(x) = \begin{cases} 1 & \text{if } x = 0 \\ x & \text{if } x \in (0, 1] \end{cases}$$

Assume by contradiction that for any $\epsilon > 0$, there exists a depth-2 ReLU network N such that:

$$\sup_{x \in [0, 1]} |f(x) - N(x)| < \epsilon$$

This implies:

$$(1) \text{ At } x = 0: |f(0) - N(0)| < \epsilon \implies |1 - N(0)| < \epsilon$$

$$(2) \text{ For } x \in (0, 1]: |f(x) - N(x)| < \epsilon \implies |x - N(x)| < \epsilon$$

We know from the hint that ReLU networks are continuous functions in all of $\mathbb{R}$. In particular:

$$\lim_{x \rightarrow 0^+} N(x) = N(0)$$

Therefore,

$$\lim_{x \rightarrow 0^+} |x - N(x)| \leq \epsilon \implies |0 - N(0)| \leq \epsilon \implies |N(0)| \leq \epsilon$$

From (1), we have $N(0) > 1 - \epsilon$. From (2), we have $N(0) \leq \epsilon$.

Combining these:

$$1 - \epsilon < \epsilon \implies 1 < 2\epsilon \implies \epsilon > \frac{1}{2}$$

This contradicts the assumption that the inequality holds for *every* $\epsilon > 0$ (specifically for $\epsilon \leq 0.5$). For example we can take $\epsilon = \frac{1}{4}$ and get from (1) that $N(0) > \frac{3}{4}$ and from (2) we get $N(0) \leq \frac{1}{4}$. **Contradiction.**

Thus, $f(x)$ cannot be approximated by $N(x)$ under the supremum norm.

2. Variation of UAT using L^1 approximation

1. Note

Since f is bounded with finitely many discontinuities, f is Riemann integrable on $[0, 1]$ so for any $\epsilon > 0$, there exists a step function:

$$S(x) = \sum_{j=1}^m c_j \mathbf{1}_{(a_{j-1}, a_j)}(x)$$

where $0 = a_0 < a_1 < \dots < a_m = 1$, such that $|f(x) - S(x)| < \frac{\epsilon}{2}$. Therefore,

$$\int_0^1 |f(x) - S(x)| dx < \frac{\epsilon}{2}$$

2. Trapezoidal Construction

Define:

$$T_{a,b}^\delta(x) = \frac{1}{\delta} ((x - a + \delta)_+ - (x - a)_+ - (x - b)_+ + (x - b - \delta)_+)$$

Proof:

- For $x < a - \delta$: all ReLU terms are 0, so $T_{a,b}^\delta(x) = 0$.
- For $x \in (a - \delta, a)$: the value is $\delta T = (x - a + \delta)_+ = x - a + \delta$. So, $T_{a,b}^\delta(x) = \frac{1}{\delta}(x - a + \delta)$, which increases linearly from 0 to 1.
- For $x \in (a, b)$: $\delta T = (x - a + \delta) - (x - a) = \delta \rightarrow T = 1$.
- For $x \in (b, b + \delta)$: $\delta T = \delta - (x - b)_+ = \delta - x + b$. So, $T_{a,b}^\delta(x) = \frac{1}{\delta}(-x + b + \delta)$. This function decreases linearly from 1 to 0.
- For $x > b + \delta$: we get $\delta T = \delta - (x - b) + (x - b - \delta) = \delta - \delta = 0$.

To summarize we got:

$$T_{a,b}^\delta(x) = \begin{cases} 1 & \text{if } x \in [a, b] \\ 0 & \text{if } x \in (a - \delta, b + \delta) \end{cases}$$

So we have shown how to create the trapezoid using four ReLU units as required.

The distance bound:

Denote,

$$d = \int_0^1 |1_{[a,b]}(x) - T_{a,b}^\delta(x)| dx$$

Notice that while $x < a - \delta$, both functions are 0, and similarly for $x > b + \delta$. We'll use $\Delta = 1_{[a,b]} - T_{a,b}^\delta$ as a notation. Therefore, we get:

$$d = \int_{a-\delta}^a |\Delta| dx + \int_a^b |\Delta| dx + \int_b^{b+\delta} |\Delta| dx$$

Between $a - \delta$ and a , the indicator is 0, so $|\Delta| = T_{a,b}^\delta(x)$. This is also the case for the region between b and $b + \delta$. Between a and b both functions equal 1, so $|\Delta| = 0$. Therefore,

$$\begin{aligned} d &= \int_{a-\delta}^a T_{a,b}^\delta(x) dx + \int_b^{b+\delta} T_{a,b}^\delta(x) dx \\ \implies d &= \frac{1}{\delta} \int_{a-\delta}^a (x - a + \delta) dx + \frac{1}{\delta} \int_b^{b+\delta} (\delta - x + b) dx \end{aligned}$$

Both integrals are areas of triangles with base δ and height 1, therefore each triangle contributes $\frac{\delta}{2}$ to the total L^1 distance. In other words,

$$d = \frac{\delta}{2} + \frac{\delta}{2} = \delta$$

So, we have shown that:

$$\int_0^1 |1_{[a,b]}(x) - T_{a,b}^\delta(x)| dx = \delta$$

That means that the L^1 distance between the indicator and the trapezoid is given by δ , and we can pick δ to be arbitrarily small.

3. Network Construction

Let $S(x) = \sum_{j=1}^m c_j \mathbf{1}_{[a_{j-1}, a_j]}(x)$ be a step function we've seen before, such that $\int_0^1 |f(x) - S(x)| dx < \frac{\epsilon}{2}$. We define the network $N(x)$ as:

$$N(x) = \sum_{j=1}^m c_j T_{a_{j-1}, a_j}^\delta(x)$$

where each T^δ is the trapezoidal function defined in Part 2.

We can see that each $T_{a,b}^\delta(x)$ is composed of 4 ReLU units. Therefore, $N(x)$ is a linear combination of $4m$ ReLU units. This corresponds to a depth-2 network with $4m$ hidden neurons (width) and a single linear output unit.

By the triangle inequality we get :

$$\int_0^1 |S(x) - N(x)| dx \leq \int_0^1 \sum_{j=1}^m |c_j| |\mathbf{1}_{[a_{j-1}, a_j]}(x) - T_{a_{j-1}, a_j}^\delta(x)| dx$$

Notice that the integral of a sum can be swapped with the sum of integrals like that:

$$\int_a^b (f(x) + g(x)) dx = \int_a^b f(x) dx + \int_a^b g(x) dx$$

We can write:

$$\int_0^1 |S(x) - N(x)| dx \leq \sum_{j=1}^m |c_j| \int_0^1 |\mathbf{1}_{[a_{j-1}, a_j]}(x) - T_{a_{j-1}, a_j}^\delta(x)| dx$$

Substituting the result from Part 2 ($\int |\Delta| = \delta$):

$$\int_0^1 |S(x) - N(x)| dx \leq \delta \sum_{j=1}^m |c_j|$$

To satisfy the proof, we choose a sufficiently small δ such that:

$$\delta < \frac{\epsilon}{2 \sum_{j=1}^m |c_j|}$$

That implies:

$$\int_0^1 |S(x) - N(x)| dx < \frac{\epsilon}{2}$$

4. Final Conclusion

We now combine everything to prove the Universal Approximation Theorem inequality for the L^1 norm:

$$\int_0^1 |f(x) - N(x)|dx = \int_0^1 |f(x) - S(x) + (S(x) - N(x))|dx$$

Using the triangle inequality and the monotonicity of integrals:

$$\int_a^b |u + v|dx \leq \int_a^b |u|dx + \int_a^b |v|dx$$

Thus:

$$\int_0^1 |f(x) - N(x)|dx \leq \int_0^1 |f(x) - S(x)|dx + \int_0^1 |S(x) - N(x)|dx$$

In Step 1, we got:

$$\int_0^1 |f(x) - S(x)|dx < \frac{\epsilon}{2}$$

And from Step 2:

$$\int_0^1 |S(x) - N(x)|dx < \frac{\epsilon}{2}$$

Combining these inequalities gives us the required UAT inequality:

$$\int_0^1 |f(x) - N(x)|dx < \frac{\epsilon}{2} + \frac{\epsilon}{2} = \epsilon$$

Thats complete the proof.

■

Part 1 - Training a CNN on SVHN

Analysis

The main weakness of the model is confusion between visually similar digits (such as 3 and 9). This is mainly due to low image quality, blur, and complex backgrounds in the SVHN dataset. In addition, the model was trained from scratch and without data augmentation, which limits its ability to learn robust and invariant visual features. As a result, the model struggles with ambiguous or low-quality digit images.

Part 2 - Adversarial Attacks on our Model

Dry Question

The transferability of adversarial examples is important because it enables attacks on models whose architecture and parameters are unknown. As demonstrated in the slide, adversarial images crafted using FGSM on one model (e.g., ResNet-50) often significantly reduce the accuracy of other models with different architectures. This property allows an attacker to perform a black-box attack by generating adversarial examples on a surrogate model and then transferring them to the target model, without requiring access to the target model's gradients or internal structure.

Different ϵ 's Test and Visualization

Based on the visual inspection of the perturbed images, the effect of the adversarial noise becomes noticeable already for small values of ϵ . While the digits are still mostly recognizable for $\epsilon \leq 0.02$, around $\epsilon \approx 0.05$ the images become significantly distorted and difficult to classify reliably by the human eye. For larger values such as $\epsilon = 0.1$, the digit structure is heavily corrupted and often no longer interpretable.

Part 3 - Adversarial Training

Discussion: Performance Comparison

The results demonstrate a significant shift in model robustness following adversarial training. While the standard model is highly accurate on clean data, it lacks stability under gradient-based perturbations.

Comparison of ResNet18 performance on SVHN		
Training Method	Clean Accuracy	FGSM Accuracy ($\epsilon = 0.1$)
Standard (Baseline)	$\approx 92\%$	$\approx 3\%$
Adversarial Training	–	$\approx 78\%$

Key Findings

- **Vulnerability:** The baseline model’s performance plummeted from 92% to 3%. This confirms that standard Empirical Risk Minimization (ERM) leads to an over-reliance on non-robust, superficial features.
- **Defense Effectiveness:** By shifting to a min-max optimization (Adversarial Training), the model achieved 78% accuracy under attack. This indicates a successful transition to learning structural features of digits.

Conclusion

Adversarial training acts as a powerful regularizer, forcing the network to prioritize features that remain invariant within an ϵ -ball of 0.1, thus ensuring reliability in hostile environments.

Part 4 - Contrastive Learning

Dry Questions

1. Batch Size

When training an unsupervised contrastive learning model such as SimCLR, a large batch size is preferred. This is because, for each positive pair in a batch of size N , there are $2(N - 1)$ negative examples. Using a larger batch increases the number of negative samples, which improves the quality of the contrastive objective and leads to better representations.

2. Embeddings Creation

When creating embeddings for images in the test set, the process differs from training in that no data augmentations are applied. During training, multiple augmented views of each image are generated in order to form positive pairs. In contrast, at test time, each image is passed only once through the encoder to obtain a single, deterministic embedding. Additionally, the projection head is typically discarded, and only the encoder representations are used.

3. Image Augmentations

- Randomly cropping a fixed-size window is desirable, as it preserves the semantic content of the image while encouraging spatial invariance.
- Enlarging the image to 128×128 is not desirable, since it does not introduce meaningful variation between views.
- Random rotation of the image is generally not desirable, as it may change the semantic meaning of objects in natural images.
- Adding Gaussian noise is desirable, as it improves robustness and encourages the model to focus on high-level features.
- Randomly changing the image dimensions is not desirable, since it distorts the image geometry and semantic structure.
- Randomly converting the image to grayscale is desirable, as it reduces reliance on color information and promotes shape-based representations.

Evaluation

Q1

Two-class separation (t-SNE):

Yes—there is a fairly clear separation between the two chosen classes. Most points form two distinct clusters (one mostly higher, the other lower), with a small amount of overlap/outliers.

Q2

Nearest neighbors (within-batch, MSE):

Partially. Some retrieved “closest” images don’t look semantically similar; the matches seem driven more by low-level cues (colors, textures, background/layout) than by object category. Likely reasons: the encoder is trained from scratch with limited epochs, using the projection-head embeddings for retrieval, and restricting search to a single batch (noisy).